



Cost-efficient quantum cloud task offloading with quantum-inspired particle swarm optimization

Santanu Ghosh , Pratyay Kuila *

Department of Computer Science & Engineering, National Institute of Technology Sikkim, Ravangla, 737139, Sikkim, India

ARTICLE INFO

Keywords:

Quantum cloud computing
Quantum task offloading
Load balancing
Energy
Delay
Price

ABSTRACT

Quantum cloud computing (QCC) empowers application users (AUs) to manage computationally intensive and resource-demanding applications, particularly those involving intractable and complex problems. This research focuses on quantum task offloading (QTO) within the QCC environment. Successful QTO decisions require careful consideration of energy consumption, execution delay, service cost, and load balancing. Incorporating task urgency, the quantum task offloading problem (QTOP) is mathematically formulated to prioritize the execution of urgent tasks while satisfying budget and deadline constraints. It is shown that QTOP is a non-deterministic polynomial-time (NP-complete) problem. To address this challenge, a quantum-inspired particle swarm optimization (QPSO) algorithm is proposed. A novel quantum particle (QP) encoding scheme is introduced and decoded using a linear hashing approach to generate valid task offloading solutions. An effective fitness function is designed by integrating two penalty variables to eliminate infeasible solutions that violate resource and budget constraints. Extensive simulations are conducted to evaluate the performance of QPSO against several baseline algorithms, where QPSO consistently outperforms the others. Furthermore, the proposed cost model is benchmarked against existing models, demonstrating superior efficiency. Statistical analysis, as well as exploration and exploitation behavior analysis, further validate the robustness of the proposed method.

1. Introduction

Quantum computing (QC) has the capability to handle complex problems that are intractable or impractical even for the most advanced supercomputers [1]. Locally executing tasks that require quantum computation can be both technically challenging and financially burdensome. As a result, it is often more practical and economical to delegate such tasks to remote infrastructure, such as cloud-based quantum servers, where the necessary quantum resources are readily available. This necessitates the integration of quantum computing with cloud technologies. The resulting paradigm, known as Quantum Cloud Computing (QCC), enables seamless access to quantum processing capabilities for end users via cloud platforms [2,3]. QCC represents a groundbreaking computational approach that leverages QC in the cloud to solve complex problems for users with unmatched speed and efficiency. Although in the development phase, QCC stands out for its ability to perform efficiently even in isolated environments [3]. Recognizing its potential, numerous countries are investing heavily in QCC, with the support of both government initiatives and private enterprises [4]. Several leading companies, including IBM, Google, Xanadu, Amazon, Rigetti Computing, Microsoft, Oxford Quantum Circuits, QC Ware, Alibaba Group, etc.,

are actively advancing QCC through extensive research and development efforts [5,6]. As it is not possible to directly deploy quantum applications (QAs) on a remote and isolated quantum computer, they must be hosted on a traditional cloud server (TCS) [2]. Furthermore, it is possible to provide these applications as a service [7]. QA is placed in the TCS, which receives task requests from application users (AU). For every incoming request, QA will generate a matching quantum task (QTsk) consisting of one or more quantum circuits. Quantum task offloading (QTO) is the process of selecting a quantum node (QN_d), present in the TCS, to execute a QTsk. The QA translates the execution results into a format readable by the AUs and forwards them to the respective AU [8]. Fig. 1 illustrates the QTO process in QCC. Recent research highlights the abilities of QC to address complex problems in various areas, including healthcare, chemistry, and physics [9–11].

1.1. Motivation

The advent of QC is expected to reshape the landscape of cloud computing [3,12]. Despite its promise, resource optimization in QCC remains a significant challenge. Effective QTO is critical to ensure reliable and high-performance QC by selecting the appropriate hardware

* Corresponding author.

E-mail addresses: santanu1109ghosh@gmail.com (S. Ghosh), pratyay_kuila@yahoo.com (P. Kuila).

<https://doi.org/10.1016/j.future.2025.108095>

Received 28 February 2025; Received in revised form 30 June 2025; Accepted 18 August 2025

Available online 22 August 2025

0167-739X/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

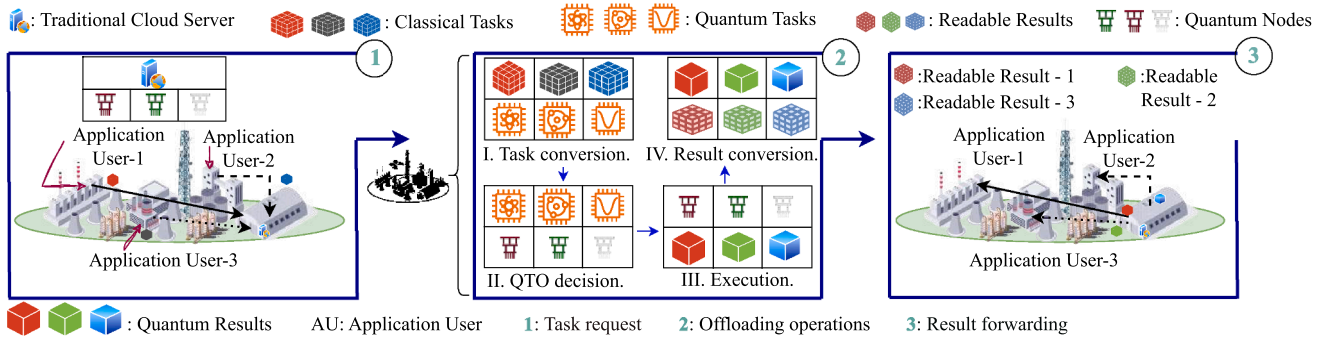


Fig. 1. Quantum Cloud Computing.

and configurations [7]. The delay comes as an important factor that needs to be minimized for a quick response [2]. Moreover, the study in Ravi et al. [12] emphasizes that QCC environments experience significant queuing delays, which are expected to grow with increasing user demand. Hence, a well-designed QTO strategy is vital to reducing waiting times and improving responsiveness. In addition to latency, energy consumption is another crucial factor that must be considered during task offloading (TO) [13]. Given that quantum computations are inherently resource-intensive [3,14], cost or pricing must also be optimized as part of the offloading strategy. Similar to traditional TO, load balancing on QNs will ensure that QTsks are distributed properly while performing the QTO [15]. Proper load distribution not only minimizes system bottlenecks but also significantly reduces overall waiting time, thereby enhancing system-wide efficiency [12].

The quantum task offloading problem (QTOP) is classified as non-deterministic polynomial (NP)-complete. In this context, metaheuristic algorithms (MAs) such as particle swarm optimization (PSO), genetic algorithm (GA), etc., have gained significant attention for their potential to efficiently generate near-optimal solutions for complex NP-complete problems within polynomial time. The rapid exploration capabilities of quantum computation, in conjunction with MAs, result in quantum-inspired MAs (QMAs) [13]. QMAs are frequently used to solve challenging optimization problems. The quantum-inspired PSO (QPSO) [16] is a highly effective approach for addressing complex problems.

1.2. Author's contribution

The contributions of this research are as follows:

- QPSO is proposed for the QTOP in QCC with multiple users. The problem is first mathematically formulated, considering the urgency of QTsks, the overall energy consumption, delay, load balance, and price. It has been demonstrated that the QTOP is NP-complete.
- A novel quantum encoding scheme is employed to represent quantum particles (QPs), where each group of components within a QP corresponds to a specific QTsk. These QPs are then decoded using a linear hashing technique to generate a complete and valid solution to the task offloading problem.
- An appropriate fitness function is formulated by taking into account energy consumption, execution delay, service cost, and system load balancing. To ensure deadline compliance, a penalty mechanism is integrated into the function to discard decoded particles (DPs) that violate QTsk execution constraints. It is also demonstrated that all phases of the QPSO algorithm can be executed within polynomial time.
- Extensive simulation is done on multiple scenarios, and the performance of QPSO is compared with GA, PSO, gravitational search algorithm (GSA), differential evolution (DE), and DRL [2]. A statistical analysis was also performed. Moreover, the algorithm shows better exploration and exploitation.

1.3. Organization of the paper

The related works are surveyed in Section 2. Section 3 presents the models, notations, and problem formulation. Section 4 offers an overview of PSO. While, Section 5 highlights the proposed quantum-inspired PSO (QPSO), followed by results and analysis in Section 6. Section 7 brings the work to a conclusion. Tables 1 and 3 depict the list of abbreviations and symbols used in the entire research.

2. Related works

QCC is an emerging field of research focused on solving complex problems through the internet [17]. Section 2.1 provides a detailed discussion on QCC. However, unlike traditional task offloading (TTO), QTO's development is still in its infancy. To provide a comprehensive understanding of the existing research, a detailed review has been conducted in Sections 2.2–2.3, and the findings are summarized in Table 2 below to highlight the key insights.

2.1. Quantum cloud computing

QCC enables remote access to quantum computing resources through the cloud, allowing users to execute quantum algorithms and simulations without the need to own or maintain quantum hardware. Similar to traditional cloud computing, QCC is categorized into three primary service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). In IaaS, users have complete control over a quantum device, including the ability to select and manipulate specific qubits for their computations [18]. IBM Quantum [19] and Quantum Inspire [20] are two examples of IaaS. In contrast, PaaS abstracts the complexities of hardware and software management, offering developers an integrated environment for building, deploying, and managing quantum applications. This model, also referred to as Quantum Computing as a Service (QCaaS) or Quantum as a Service (QaaS), supports a typical workflow that involves installing the development platform, designing quantum circuits, selecting appropriate execution backends, and running the circuits [8]. A further evolution of this model is Quantum Algorithms as a Service (QAaaS), which simplifies the development of hybrid quantum-classical applications by concealing quantum-specific details, allowing developers to focus solely on application logic [17]. Microsoft Azure Quantum [21] is one of the examples of the PaaS approach. Lastly, the SaaS model provides end-users with access to ready-to-use quantum applications via the cloud, without requiring technical expertise or infrastructure management. An example of this is D-Wave's quantum hybrid solver, which allows users to solve complex problems using quantum resources through a web-based interface [22]. Recently, the use of SaaS and PaaS models for allocating QNs to AUs to execute their QTsks has emerged as a promising and increasingly studied area. This process, known as QTO, refers to the delegation of quantum computational tasks from AUs to more capable

Table 1
Frequently used abbreviations.

LightGreen	Short form	Full Form	Short form	Full Form
AU		Application user	QAaaS	Quantum algorithms as a service
BP		Binary particle	QaaS	Quantum as a service
CLOPS		Circuit layer operation per second	QC	Quantum computing
CX		Controlled NOT	QCaaS	QC as a service
DE		Differential evolution	QCC	Quantum cloud computing
DP		Decoded particle	TTO	Traditional TO
DQC		Distributed QC	QGSA	Quantum-inspired GSA
DRL		Deep RL	QMA	Quantum-inspired MA
GA		Genetic algorithm	QNd	Quantum node
GSA		Gravitational search algorithm	QP	Quantum particle
HGP		Hybrid GA and PSO	QPSO	Quantum-inspired PSO
IaaS		Infrastructure as a service	QPU	Quantum processing unit
KP		0–1 Knapsack problem	QTO	Quantum TO
MA		Metaheuristics algorithms	QTOP	QTO problem
MEC		Mobile edge computing	QTsk	Quantum task
PaaS		Platform as a service	RL	Reinforcement learning
PSO		Particle swarm optimization	TCS	Traditional cloud server
QA		Quantum application	TO	Task offloading

Table 2

Collective survey (RAP: resource allocation problem, TTO: traditional task offloading, QTO: quantum task offloading, QUA: qubit allocation, QCC: quantum cloud computing, MEC: mobile edge computing, FC: fog computing, AU: application users, MA: metaheuristics algorithm, QMA: quantum-inspired metaheuristics algorithm, DQC: distributed quantum computing, LB: load balance, TU: task urgency).

Reference	RAP	Environment	AU	Delay	Energy	Price	LB	TU	Technique
El Menbawy et al. [28]	TTO	FC	Multiple	✓	✓	✗	✗	✗	HGP (MA)
Nguyen et al. [2]	QTO	QCC	Multiple	✓	✗	✗	✗	✗	DRL
Ghosh and Kuila [15]	TTO	MEC	Multiple	✓	✓	✗	✓	✗	GSA (MA)
Ravi et al. [23]	QTO	QCC	Multiple	✓	✗	✗	✗	✗	Heuristics
Ghosh et al. [13]	TTO	MEC	Multiple	✓	✓	✓	✗	✗	QGSA (QMA)
Kaewpuang et al. [24]	QUA	QCC	Multiple	✓	✗	✗	✗	✗	Heuristics
Ghosh et al. [26]	TTO	MEC	Multiple	✓	✓	✗	✗	✗	PSO (MA)
Ngoenriang et al. [25]	DQC	QCC	Multiple	✗	✗	✓	✗	✗	Stochastic programming
Anqing and Youyun [27]	TTO	MEC	Single	✓	✓	✗	✗	✗	GA (MA)
QPSO (Proposed)	QTO	QCC	Multiple	✓	✓	✓	✓	✓	QPSO (QMA)

quantum cloud infrastructures. Although still in its nascent stage, QTO holds significant potential for enhancing the efficiency and scalability of quantum applications. Previous research efforts on QTO are reviewed in Section 2.2.

2.2. Quantum task offloading

Nguyen et al. [2] devised an offloading strategy using deep reinforcement learning (DRL) by considering computation and waiting time. An adaptive task scheduling strategy was put out by Ravi et al. [23] using a simple statistical analysis of historical data. Kaewpuang et al. [24] presented a novel qubit allocation approach tailored to quantum cloud environments. This method considers the uncertainty of quantum circuit requirements and expected waiting times, dividing the strategy into two phases: (i) On-Demand Phase: Focused on real-time resource requirements. (ii) Reservation Phase: Based on historical data to guide resource allocation. Ngoenriang et al. proposed a resource allocation strategy for distributed quantum computing (DQC). This method leverages stochastic programming to minimize the total cost associated with quantum task offloading [25].

2.3. Traditional task offloading

Ghosh et al. addressed the Task offloading (TO) problem (TOP) by incorporating delay, energy, and price. The TOP was solved utilizing the quantum-inspired gravitational search algorithm (QGSA) [13]. In another study, Ghosh and Kuila [15] demonstrated the application of the gravitational search algorithm (GSA) to solve TOP, focusing on load, energy, and delay minimization, with an emphasis on providing effective solutions for humanitarian relief operations. Additionally, Ghosh et al.

[26] employed PSO to solve TOP, specifically minimizing delay and energy. Anqing and Youyun [27] addressed the TOP problem in a mobile edge computing (MEC) system. They employed an improved genetic algorithm (GA) to optimize the process, focusing on minimizing delay and energy consumption. Another research on TOP, focusing on delay and energy minimization, is presented in El Menbawy et al. [28]. The study employs an improved hybrid GA and PSO (HGP), to achieve efficient optimization.

While QTO and TTO both aim to solve task offloading problems, QTO focuses on allocating and optimizing resources for quantum computing workloads, whereas TTO addresses the allocation of classical computing resources in a traditional or edge computing context. This distinction guides the choice of algorithm and its application. These studies collectively provide valuable insights into the emerging field of QTO, emphasizing various strategies to minimize delay, reduce energy consumption, and minimize price and load in quantum computing environments.

2.4. Research gaps and novelties

A summary of the related works with research gaps is provided in Table 2. Furthermore, the presented work offers below advantages over the existing works.

- This study employs QMAs, which require significantly less computational time compared to the DRL / RL-based methods. Despite the efficiencies of QMAs, the potential application of the same in QCC has remained unexplored.
- When measuring task priority or urgency during QTO, it is important to take into account how long the tasks must take to complete in order to prevent missing the task execution deadline. However,

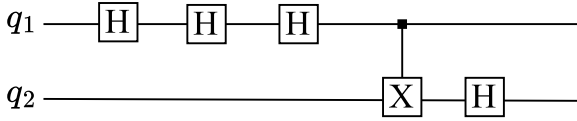


Fig. 2. Quantum circuit.

when determining task urgency, the studies in Nguyen et al. [2], Ravi et al. [23], Kaewpuang et al. [24], Ngoenriang et al. [25] ignore this crucial component. This work, on the other hand, presents a unique method that incorporates the necessary execution time of the QTsks to assess task urgency.

- Quantum computing operations are inherently costly, yet previous research on QCC [2,23–25] has largely overlooked this aspect. This study addresses this gap by incorporating task execution price into the decision-making process for efficient QTO.
- Quantum computing operations are highly energy-intensive, yet previous research on QCC [2,23] has not considered this critical factor. This study bridges the gap by integrating energy consumption into the decision-making process for more efficient QTO.
- The research on QTO presented in Nguyen et al. [2], Ravi et al. [23] does not incorporate multiple factors such as energy, price, or load balancing in making QTO decisions.

3. Terminologies, models, and problem formulation

3.1. Terminologies

In QCC, a quantum computer can be accessed as a QNd. Each QNd is supported by a quantum processing unit (QPU). The QPUs help in processing the QTsks, which come in the form of quantum circuits [29]. QPU consists of gates that conduct computation on one or more qubits. Below are some terminologies that are associated with QCC.

Definition 3.1 (Qubit). The smallest unit of quantum information is called a qubit, which can exist in the basis states $|0\rangle$, $|1\rangle$, or in any linear superposition of these states, expressed as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex coefficients [13]. A qubit can be represented as a column vector $q = [\alpha \ \beta]^T$, with the normalization condition $|\alpha|^2 + |\beta|^2 = 1$.

In Fig. 2, q_1 and q_2 represent the two qubits being manipulated. These qubits undergo operations defined by the quantum gates in the circuit.

Definition 3.2 (Multi-Qubit). A multi-qubit system consisting of D qubits is mathematically expressed as Ghosh et al. [13]:

$$\bar{Q} = [q_1, \dots, q_m, \dots, q_D] = \begin{bmatrix} \alpha_1 & \dots & \alpha_m & \dots & \alpha_D \\ \beta_1 & \dots & \beta_m & \dots & \beta_D \end{bmatrix} \quad (1)$$

where, $\alpha_m^2 + \beta_m^2 = 1, \forall m = \{1, 2, \dots, D\}$.

Illustration 3.1. Let there be a four-qubit system:

$$Q = \begin{bmatrix} \frac{-1}{2} & \frac{1}{\sqrt{2}} & -\frac{\sqrt{3}}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{-1}{\sqrt{2}} & \frac{-1}{2} & \frac{-1}{2} \end{bmatrix}$$

. There are $2^4 = 16$ number of states as: $|0000\rangle, |0001\rangle, \dots, |1111\rangle$. The probability measure of the given state $|1010\rangle$ is determined as $(\frac{-\sqrt{3}}{2})^2 \times (\frac{1}{\sqrt{2}})^2 \times (\frac{-1}{2})^2 \times (\frac{\sqrt{3}}{2})^2 = \frac{9}{128}$.

Definition 3.3 (Quantum Circuit). It is a basic framework of quantum operations and gates, and is used to model and execute quantum algorithms [30]. Fig. 2 depicts a quantum circuit consisting of two qubits (q_1, q_2), a controlled-NOT (CX) gate, and Hadamard gates. In this circuit, the qubit q_1 undergoes three successive Hadamard operations (H). After that, q_1 acts as the control qubit, and q_2 is the target qubit in CX. Finally, q_2 experiences a H gate.

Definition 3.4 (Quantum Volume). Quantum volume is a single metric that quantifies the quantum computational capacity of a QPU. It takes into account both the number of qubits and the quality of quantum operations [31].

If a QPU has a quantum volume of 2^ℓ , then it can reliably execute a square quantum circuit with ℓ qubits and ℓ layers of random two-qubit gates. A higher quantum volume means the QPU can reliably execute larger and more complex quantum circuits.

Definition 3.5 (Circuit Depth). The circuit depth refers to the number of sequential computational steps required to execute a quantum circuit, corresponding to the length of the longest path from input to output. Each gate operation in this path is counted as a single unit, and the total depth is always represented as an integer. When multiple gates are executed on qubits in parallel, only a single step is counted. In contrast, gates applied sequentially on the same qubit increase the depth accordingly.

As shown in Fig. 2, the circuit can be divided into five steps. The first three steps apply consecutive Hadamard gates to qubit q_1 , followed by a CX (controlled-NOT) gate in the fourth step, and another Hadamard gate in the fifth step. Thus, the depth of the circuit is 5.

Definition 3.6 (Circuit Layer Operation Per Second). The number of quantum circuit layers a QPU can execute per time unit is known as its circuit layer operation per second (CLOPS) [30].

Definition 3.7 (Qubit Topology). The hardware's qubit layout is represented by the qubit topology graph of a device. It shows how the physical qubits are coupled in a device [32]. For the circuit in Fig. 2, the CX gate between q_1 and q_2 indicates that these qubits are physically or logically connected.

Definition 3.8 (Quantum Task). Within the framework of a larger traditional computer program, a quantum task (QTsk) is a more comprehensive idea of a program or algorithm that runs or samples from several quantum circuits [29,30]. The circuit in Fig. 2 could represent a single quantum task within a larger quantum algorithm, illustrating how qubits interact through a sequence of gates.

Definition 3.9 (Number of Shots). It means the number of times a QTsk in the form of an algorithm or a program is executed [2]. For the circuit in Fig. 2, the number of shots indicates how many times the operations on q_1 and q_2 are repeated to ensure reliable measurement outcomes.

3.2. System model

In this study, a Quantum Cloud Computing (QCC) environment is considered, wherein tasks are initially transmitted from application users (AUs) to a cloud server in their classical form, as illustrated in Fig. 3. The cloud server hosts quantum nodes (QNds) equipped with quantum processing units (QPUs) to handle these tasks. Upon receipt, each task is transformed into its corresponding quantum task (QTsk). A quantum task offloading (QTO) decision is then made to determine the most suitable QNd for executing each QTsk. After execution, the quantum results are converted back into a classical format and returned to the respective AUs.

This work primarily focuses on the QTO decision process. The system is assumed to consist of M AUs, each generating a task that requires quantum computation. These task requests are sent directly to a quantum application (QA) deployed on a traditional cloud server (TCS). The QA converts each classical request into a QTsk, resulting in a total of M QTsks. The TCS comprises N QNds, each integrated with a dedicated QPU. Every QTsk must be executed on one of these QNds. If multiple QTsks are assigned to the same QNd, they are processed sequentially. The execution of a QTsk depends on the QNd having a sufficient number of qubits to perform the required computation. Additionally, each QTsk must be completed within a specified time threshold. The following terminology is used to describe the proposed model in detail. The detailed terminologies are given in Table 3.

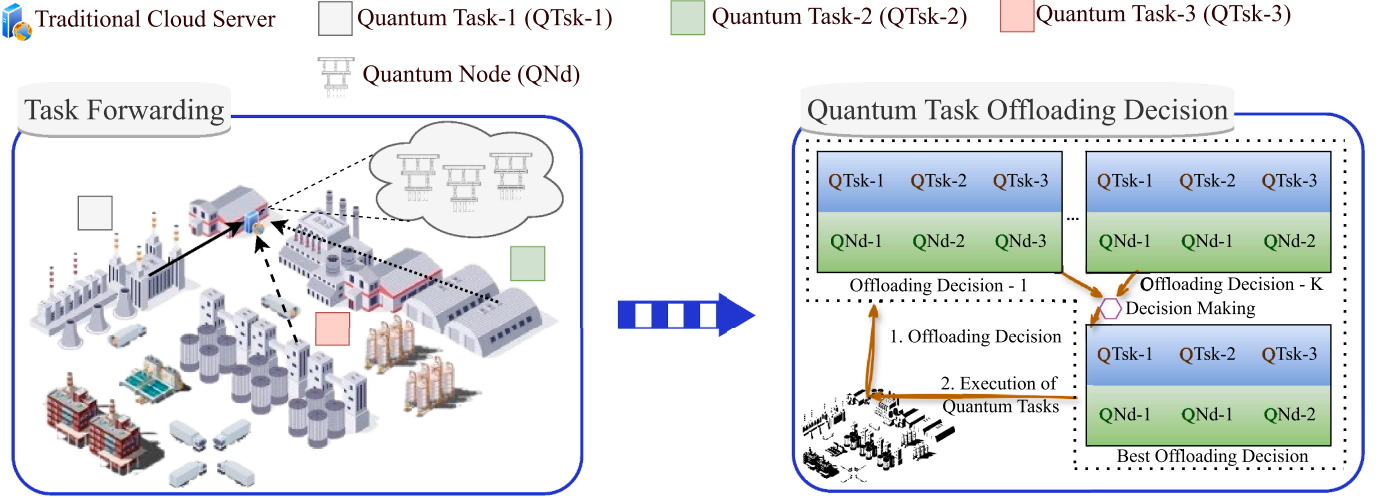


Fig. 3. System Model.

Table 3
Frequently used symbols and their meaning.

Symbol	Representation	Symbol	Representation
ϕ_g	The g th quantum task (QTsk)	ϕ_g^q	Number of qubits in QTsk ϕ_g
ϕ_g^d	Circuit depth of QTsk ϕ_g	ϕ_g^p	Threshold price of ϕ_g
ϕ_g^T	Threshold time of ϕ_g	$\xi_{g,j}$	Decision variable as in Eq. (2)
δ_g	Total delay to execute ϕ_g	δ_j	Total execution delay on QNd n_j
δ^w	Waiting delay	σ_j	Load of the QNd n_j
ρ_j	Price per second charged by n_j	π	Permutation of the indices $\{1, 2, \dots, M\}$
$\Theta_i^p(t)$	Personal best angles of $\bar{Q}_i$ at i th generation	$\bar{\Theta}^s(t)$	Global best angles at i th generation
$b_{i,m}$	Binary observed value of qubit $q_{i,m}$	B_i	The i th binary particle
δ'	Total delay	D	Dimension of QP
e^c	Computation energy	e'	Total energy
$\bar{Q}_i$	Angular form of QP $\bar{Q}_i$	A_i	The i th decoded particle
σ^m	Maximum load	P_{size}	Population size
M	Number of QTsks/AUs	u_g	The g th application user (AU)
$\mathbb{P}^m$	Maximum price	n_j^q	Scale of n_j
n_j^q	Quantum volume of n_j	o_g	The number of QNds options for ϕ_g
N	Number of quantum nodes (QNds)	n_j	The j th QNd
$\mathbb{P}_g$	Price for processing QTsk ϕ_g	P_i	The i th particle in PSO
QB_g	Number of qubits for ϕ_g in a QP	Q_{POP}	Quantum population
$\theta_{i,m}$	Angular position of qubit $q_{i,m}$	U_g	Task urgency of ϕ_g
UR	Urgency list	$\text{Var}(\sigma)$	Variation of system load (Eq. (12))
δ_g^w	Waiting delay for ϕ_g	ϕ_g^s	Number of shots in ϕ_g
δ^c	Computation delay	δ^m	Maximum delay
n_j^s	CLOPS of n_j	$\mathcal{O}$	Set of options
$\mathbb{P}^t$	Total price	$\bar{Q}_i$	The i th quantum particle (QP)

- The set of N QNds is denoted by $QN = \{n_1, \dots, n_j, \dots, n_N\}$. Each QNd n_j is characterized by a set of features or resources, including the number of qubits (n_j^q), qubit topology (n_j^t), quantum volume (n_j^v), CLOPS (n_j^s), and supported gates [2,30]. It is assumed that the QNds in QN are ordered in ascending order of the number of qubits, i.e., based on n_j^q . It can be said $n_1^q \leq n_2^q \leq \dots \leq n_N^q$. Thus, it can be said that $n_1^q \leq n_2^q \leq \dots \leq n_N^q$.
- The set of M application users (AUs) is represented as $\mathbb{U} = \{u_1, \dots, u_g, \dots, u_M\}$. Each AU u_g generates a QTsk ϕ_g , and the complete set of QTsks is $\mathbb{QT} = \{\phi_1, \dots, \phi_g, \dots, \phi_M\}$. A QTsk ϕ_g is described by a five-tuple: $\phi_g = \{\phi_g^d, \phi_g^s, \phi_g^q, \phi_g^T, \phi_g^p\}$, where ϕ_g^d is the circuit depth, ϕ_g^s is the number of shots, and ϕ_g^q is the number of qubits required to execute the task. The parameter ϕ_g^T denotes the execution deadline (threshold time), and ϕ_g^p represents the maximum price the AU is willing to pay for executing ϕ_g .
- To offload a quantum task ϕ_g , a QNd $n_j \in QN$ must have sufficient qubits to support the computation, i.e., $n_j^q \geq \phi_g^q$ [2]. Not all QNds may satisfy this constraint for every task. It is assumed that each task ϕ_g has $o_g \leq N$ feasible offloading options among the available

N QNds. Furthermore, it is assumed that at least one QNd is capable of executing each task. The set of offloading options for all tasks is denoted as $\mathbb{O} = \{o_1, \dots, o_g, \dots, o_M\}$. Hence, the total number of available QNd options for all tasks is bounded by $\sum_{g=1}^M o_g \leq (N \times M)$.

- The QTO decision on ϕ_g is denoted by the Boolean variable $\xi_{g,j}$:

$$\xi_{g,j} = \begin{cases} 1, & \text{if } \phi_g \text{ is offloaded to } n_j \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Here, $\xi_{g,j} = 1$ implies that n_j has sufficient qubits to execute ϕ_g , i.e., $n_j^q \geq \phi_g^q$.

3.3. Delay model

The system delay may arise due to the following components:

- Execution or computation delay ($\delta_{g,j}^c$): The delay for executing ϕ_g entirely on n_j is determined by the following Eq. (3), as in Nguyen et al. [2]:

$$\delta_{g,j}^c = \xi_{g,j} \cdot \frac{\phi_g^d \times \phi_g^s}{n_j^s} \quad (3)$$

- Waiting delay (δ_g^w): Let n_j already have l QTsks in its execution queue, all of which have been offloaded from other AUs. Meanwhile, a new task, ϕ_g , is offloaded to n_j . As a result, ϕ_g must wait until all the previously queued tasks are fully executed on n_j . The execution times of previously queued tasks are calculated using Eq. (3). Thus, the waiting delay of ϕ_g can be estimated as [2]:

$$\delta_g^w = \sum_{r=1}^l \delta_{rj}^c \quad (4)$$

Definition 3.10. [Total time taken to execute a QTsk ϕ_g] As in Nguyen et al. [2], the total time required to complete the execution of ϕ_g is the waiting time, followed by the computation time. It can be expressed as:

$$\delta_g = \delta_{gj}^c + \delta_g^w = \delta_{gj}^c + \sum_{r=1}^l \delta_{rj}^c \quad (5)$$

where δ_{gj}^c and δ_g^w are the computation and waiting time of QTsk ϕ_g on QNd n_j as in Eqs. (3) and (4), respectively.

Definition 3.11 (Total time taken by a QNd n_j to execute all the QTsks assigned to it). When l numbers of QTsks are assigned to n_j , all the QTsks are executed one by one in n_j . The total time required to complete the execution of all the QTsks assigned to a QNd n_j can be expressed as:

$$\delta_j = \sum_{r=1}^l \delta_{rj}^c \quad (6)$$

where δ_{gj}^c denotes the computation time for QTsk ϕ_g on QNd n_j as in Eq. (3).

Definition 3.12 (Total delay (δ^t)). There are M QTsks distributed among N QNds. The total delay (δ^t) is defined as the maximum time taken by the QNds to complete the execution of all assigned QTsks as in Eq. (7).

$$\delta^t = \max\{\delta_j, \forall j \in [1, N]\} \quad (7)$$

where δ_j can be calculated using Eq. (6).

Definition 3.13 (Maximum possible delay (δ^m)). Let us assume that n_j is the QNd with the minimum CLOPS, that is, $n_j^s = \min\{n_k^s \mid 1 \leq k \leq N\}$. If all M QTsks are assigned to n_j , they will be executed sequentially, one after another. The total time required to process all QTsks at n_j is given by:

$$\delta^m = \sum_{g=1}^M \delta_{gj}^c, \quad (8)$$

where δ_{gj}^c can be calculated using Eq. (3).

3.4. Energy model

The computation energy consumption for QTsk ϕ_g is estimated by following the approach as in Parker and Vermeer [14], and is presented in Eq. (9).

$$e_g^c = \delta_{gj}^c \times \phi_g^q \times p_j \quad (9)$$

where p_j is the average power consumed per qubit. In addition to the power used directly by each qubit to carry out fundamental logic activities, the power per qubit also accounts for all power used by hardware overhead, including cooling, traditional co-processing, and other types of hardware system management.

Definition 3.14 (Total energy consumption (e^t)). It is the total energy consumption to compute all the M QTsks. It can be calculated as:

$$e^t = \sum_{g=1}^M e_g^c = \sum_{g=1}^M \{\delta_{gj}^c \times \phi_g^q \times p_j\} \quad (10)$$

where computation energy consumption e_g^c can be calculated as per Eq. (9).

Definition 3.15 (Maximum possible energy (e^m)). The maximum energy consumption is considered as the total energy consumption when all the M tasks are offloaded serially to the single QNd n_j as in Definition 3.13. It is calculated as follows:

$$e^m = \sum_{g=1}^M e_g^c \quad (11)$$

where computation energy consumption (e_g^c) of ϕ_g in n_j can be calculated as per Eq. (9).

3.5. Load balancing model

Load balancing is required to prevent QNds from getting overloaded. The load on a QNd n_j is represented as $\sigma_j = \sum_{g=1}^M \delta_{gj}^c \cdot \phi_g^d$. Mean load across all QNds is $\bar{\sigma} = \frac{\phi_{total}^d}{N} = \frac{1}{N} \sum_{g=1}^M \phi_g^d$. The variation of the load across all the QNds is as follows:

$$\text{Var}(\sigma) = \frac{1}{N} \sum_{j=1}^N (\sigma_j - \bar{\sigma})^2 \quad (12)$$

Definition 3.16 (Maximum possible load (σ^m)). It is when all the M QTsks are allocated to a QNd n_j . The maximum load (σ^m) is calculated as:

$$\sigma^m = \frac{(\sigma_j - \bar{\sigma})^2}{N}, \quad (13)$$

where the total load on n_j is $\sigma_j = \sum_{g=1}^M \phi_g^d$.

3.6. Price model

Dynamic pricing policy is a commercial strategy used in this emerging industry to provide AUs with affordable and scalable computing resources when needed [13]. This strategy is also used by QCC to assist AUs in utilizing the resources of quantum computing [5,6]. When using QPUs, IBM's standard plan costs per Qiskit runtime in seconds [5]. However, AWS charges AUs according to two separate methods: (i) per-task (which is the same for all tasks regardless of QPU hardware), and (ii) per-shot (which varies depending on QPU hardware) [6]. In this research, the QCC price is charged based on the execution time (Eq. (3)) of a QTsk ϕ_g by QNd n_j . Each QNd n_j has different execution schemes; as a result, there are different prices charged for different n_j . The price incurred while processing a QTsk ϕ_g is as follows:

$$\mathbb{P}_g = \rho_j \times \delta_{gj}^c \quad (14)$$

where, ρ_j is the price per second charged by n_j .

Definition 3.17 (Total price ($\mathbb{P}^t$)). The total price ($\mathbb{P}^t$) for all the M QTsk is calculated as:

$$\mathbb{P}^t = \sum_{g=1}^M \mathbb{P}_g = \sum_{g=1}^M \{\rho_j \times \delta_{gj}^c\} \quad (15)$$

where $\mathbb{P}_g$ can be calculated using Eq. (14) and δ_{gj}^c using Eq. (3).

Definition 3.18 (Maximum possible price ($\mathbb{P}^m$)). Consider the maximum price per second charged as $\rho^m = \max\{\rho_j \mid \forall j, 1 \leq j \leq N\}$. Accordingly, the maximum possible price ($\mathbb{P}^m$) is calculated as:

$$\mathbb{P}^m = \sum_{g=1}^M \{\rho^m \times \delta_{gj}^c\} \quad (16)$$

3.7. Task urgency model

Task urgency (U_g) represents the priority of offloading QTsks, determined by the QTsk deadline in the AU from where the QTsk is offloaded. An urgency list can be defined as $UR = \{U_1, \dots, U_g, \dots, U_M\}$, where U_g is calculated using Eq. (17).

$$U_g = \frac{1}{(\phi_g^T - \delta_{gj}^c)} \quad (17)$$

where δ_{gj}^c refers to the computation time of the QTsks ϕ_g in the QNd n_j as in Eq. (3).

Remark 3.1. Urgent tasks must be prioritized and executed first to ensure efficient task management. Let π represent a permutation of the indices $1, 2, \dots, M$ such that the urgency values are ordered as $U_{\pi(1)} \leq U_{\pi(2)} \leq \dots \leq U_{\pi(M)}$.

3.8. Problem formulation

The quantum task offloading problem (QTOP) for a multi-application user and quantum cloud computing (QCC) systems can be written as: “Given a QCC system with M application users (AUs) generating M quantum tasks (QTsks) $\mathbb{QT} = \{\phi_1, \dots, \phi_g, \dots, \phi_M\}$. There is N number of quantum nodes (QNd) $QN = \{n_1, \dots, n_j, \dots, n_N\}$ in QCC. The problem is to offload all the QTsks based on task urgency to any one of the QNd, such that total delay (δ^t), energy consumption (e^t), load balance ($\text{Var}(\sigma)$), and price ($\mathbb{P}^t$) is minimized.” The QTOP is formulated as:

$$\text{Minimize delay } (\delta^t) \quad (18)$$

$$\text{Minimize energy } (e^t) \quad (19)$$

$$\text{Minimize load } (\text{Var}(\sigma)) \quad (20)$$

$$\text{Minimize price } (\mathbb{P}^t) \quad (21)$$

Subject to:

$$\xi_{gj} \in \{0, 1\}, \forall g \in [1, M], \forall j \in [1, N] \quad (22)$$

$$\sum_{j=1}^N \xi_{gj} = 1, \forall g \in [1, M] \quad (23)$$

$$\left\{ \begin{array}{l} (\xi_{gj} = 1) \implies (n_j^q \geq \phi_g^q), \\ \forall g \in [1, M], \forall j \in [1, N] \end{array} \right. \quad (24)$$

$$\sum_{j=1}^N \sum_{g=1}^M \xi_{gj} = M \quad (25)$$

$$\delta_g < \phi_g^T, \forall g \in [1, M] \quad (26)$$

$$\mathbb{P}_g < \phi_g^P, \forall g \in [1, M] \quad (27)$$

$$1 \leq o_g \leq N, \forall g \in [1, M] \quad (28)$$

$$\sum_{g=1}^M o_g \leq (N \times M) \quad (29)$$

$$U_{\pi(1)} \leq U_{\pi(2)} \leq \dots \leq U_{\pi(M)} \quad (30)$$

Eqs. (18)–(21) reflect the four objectives of our work. Eq. (22) defines the possible values for ξ_{gj} . Eq. (23) emphasizes that each QTsk ϕ_g can be offloaded to only one QNd. Eq. (24) outlines the fact that when a QTsk ϕ_g is offloaded to a QNd n_j , it implies that the QNd n_j possesses the required number of qubits to process the QTsk ϕ_g . Eq. (25) ensures that all the M QTsks are offloaded. Eq. (26) guarantees that each QTsk execution must be completed within its specified threshold. Eq. (27) makes sure that each QTsk execution cost must not exceed its given threshold price. Each QTsk ϕ_g has a minimum of 1 and a maximum of N options for offloading, as certified by Eq. (28). All M QTsks have a maximum of $(N \times M)$ options for QTO as depicted in Eq. (29). Finally, Eq. (30) indicates that tasks are executed in increasing order of urgency.

Remark 3.2. Interestingly, the previously presented problem is a 0, 1 integer linear programming (0,1-ILP) problem with several objectives. The objective function is linear, and the offloading decision variables $\xi_{gj}, \forall j \in [1, N], \forall g \in [1, M]$ are Boolean (0 or 1).

Remark 3.3. In contrast to traditional task offloading (TTO), QTOPs involve inherent complexities arising from quantum-specific constraints, such as those captured in Eqs. (24), (28), and (29). While the high-level objective, determining whether and where to offload a task, may seem

similar, the underlying computational model, physical constraints, and performance considerations in QCC are fundamentally different. Quantum systems are governed by limited qubit availability, short coherence times, non-clonability, and probabilistic outcomes, making classical offloading strategies unsuitable. Therefore, QCC demands the development of quantum-aware offloading frameworks that explicitly incorporate the limitations and operational principles of quantum hardware.

Theorem 1. The QTOP is NP-Complete.

Proof. We have to prove these two parts: (1) QTOP is NP-Hard by a polynomial-time reduction from the well-known 0–1 Knapsack Problem (KP), and (2) QTOP is in NP. Starting from an instance of KP, which consists of a set of items, each with a weight $w_i, \forall i \in [1, M]$ and a value $v_i, \forall i \in [1, M]$, along with a knapsack capacity C , we construct a corresponding instance of QTOP in polynomial time. Specifically, for each item i in KP, we create a QTsk in QTOP with a computational load equal to w_i and a utility or benefit equal to v_i . The QNd’s capacity for QTOP is set to be the knapsack’s capacity C . Thus, selecting a subset of tasks to maximize total utility without exceeding QNd capacity directly corresponds to choosing a subset of items to maximize total value without exceeding knapsack capacity. This transformation runs in polynomial time, implying that QTOP is at least as hard as KP. Therefore, it can be said that an extension of the KP is the QTOP ($KP \leq_p QTOP$). In this instance, $\leq_p$ indicates a polynomial-time reduction. It is obvious to say that $QTOP \in NP$. As QTOP is NP-Hard and $QTOP \in NP$, QTOP can therefore be regarded as NP-complete. $\square$

4. An overview of PSO

One well-known meta-heuristic optimization method that draws inspiration from the social behavior of fish schooling and flocking of birds is the PSO [33]. In PSO, each solution, referred to as a “particle,” moves through the search space influenced by its personal best position and the global best position found by the swarm. Each particle is represented as $\vec{P}_i = \{p_{i,1}, p_{i,2}, \dots, p_{i,g}, \dots, p_{i,M}\}$, where M is the dimension of the search space, and $p_{i,g}$ is the g th component of the i th particle $\vec{P}_i$.

The velocity of $\vec{P}_i$ is updated based on its current velocity, its distance from its personal best position ($\vec{P}_{best_i} = \{pb_{i,1}, pb_{i,2}, \dots, pb_{i,g}, \dots, pb_{i,M}\}$), and the swarm’s global best position ($\vec{GBest} = \{gb_{i,1}, gb_{i,2}, \dots, gb_{i,g}, \dots, gb_{i,M}\}$). The velocity update is given by:

$$v_{i,g}(t+1) = w \cdot v_{i,g}(t) + c_1 \cdot rand_1 \cdot (pb_{i,g}(t) - p_{i,g}(t)) + c_2 \cdot rand_2 \cdot (gb_{i,g}(t) - p_{i,g}(t)) \quad (31)$$

Here, w is the inertia weight that controls exploration and exploitation, c_1 and c_2 are the cognitive and social acceleration coefficients, respectively, and $rand_1, rand_2$ are uniformly distributed random numbers in $[0, 1]$. The position of the particle $\vec{P}_i$ is then revised using its new velocity:

$$p_{i,g}(t+1) = p_{i,g}(t) + v_{i,g}(t+1) \quad (32)$$

The fitness of each particle is evaluated at every iteration to update $\vec{P}_{best_i}$ and $\vec{Gbest}$. The process is repeated until a termination criterion is met.

5. Proposed quantum-inspired PSO (QPSO)

Quantum-inspired PSO (QPSO) leverages principles of quantum computing (QC) to enhance the exploration and exploitation capability of classical PSO. QPSO incorporates quantum mechanics concepts, such as qubits and rotation gates, to represent and update the quantum states of the quantum particles. Fig. 4 highlights the working process of QPSO. QPSO consists of the following phases:

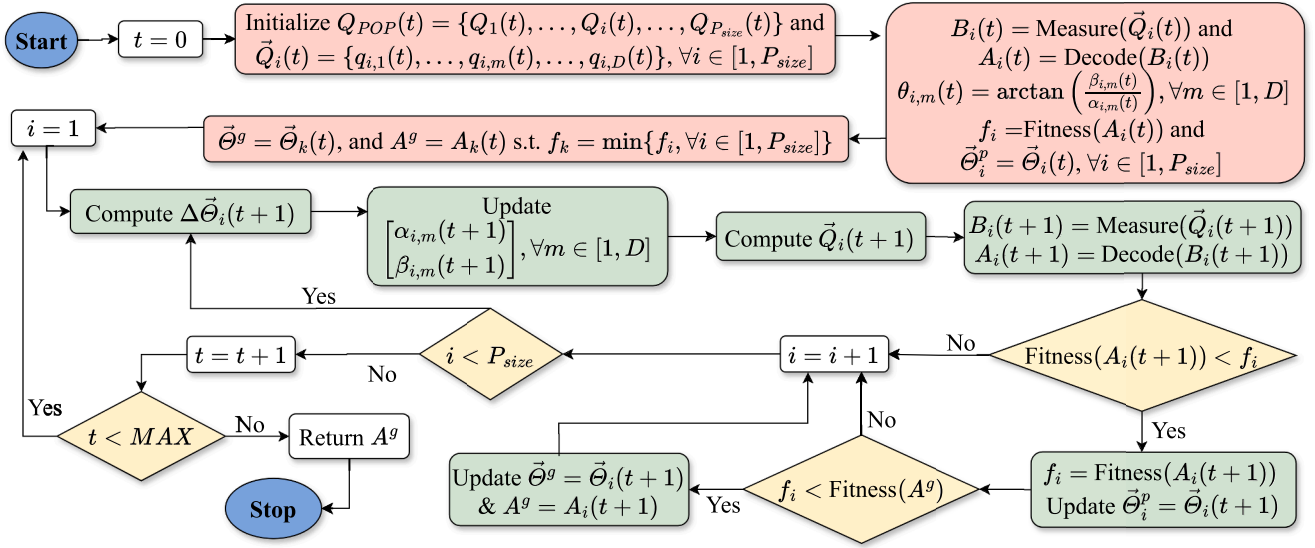


Fig. 4. Flow chart for QPSO.

Table 4

The representations of a quantum particle (QP), binary particle (BP), and decoded particle (DP) are provided for 3 QNds $\{n_1, n_2, n_3\}$ with scale $\{17, 8, 5\}$ (Section 5: Illustrations 5.1–5.3).

Task List		Task 1 (ϕ_1)		Task 2 (ϕ_2)		Task 3 (ϕ_3)		Task 4 (ϕ_4)		Task 5 (ϕ_5)		Task 6 (ϕ_6)		Task 7 (ϕ_7)	
Qubits required (ϕ^q)		ϕ_1^q 4		ϕ_2^q 6		ϕ_3^q 3		ϕ_4^q 15		ϕ_5^q 11		ϕ_6^q 7		ϕ_7^q 6	
Set of options ($\mathcal{O}$)		o_1 3		o_2 2		o_3 3		o_4 1		o_5 1		o_6 2		o_7 2	
Dimensions required		QB_1 2		QB_2 2		QB_3 2		QB_4 1		QB_5 1		QB_6 2		QB_7 2	
QP	$\vec{Q}_i(0)$	$q_{i,1}(0)$	$q_{i,2}(0)$	$q_{i,3}(0)$	$q_{i,4}(0)$	$q_{i,5}(0)$	$q_{i,6}(0)$	$q_{i,7}(0)$	$q_{i,8}(0)$	$q_{i,9}(0)$	$q_{i,10}(0)$	$q_{i,11}(0)$	$q_{i,12}(0)$		
	$\alpha_i(0)$	-0.866	0.7071	-0.866	0.5	-0.7071	-0.866	0.5	-0.5	-0.866	-0.7071	-0.866	0.5		
	$\beta_i(0)$	-0.5	0.7071	-0.5	0.866	-0.7071	-0.5	-0.866	0.866	-0.5	-0.7071	0.5	0.866		
BP	$B_1(0)$	$b_{i,1}(0)$	$b_{i,2}(0)$	$b_{i,3}(0)$	$b_{i,4}(0)$	$b_{i,5}(0)$	$b_{i,6}(0)$	$b_{i,7}(0)$	$b_{i,8}(0)$	$b_{i,9}(0)$	$b_{i,10}(0)$	$b_{i,11}(0)$	$b_{i,12}(0)$		
		0	1	1	1	1	0	1	0	0	1	1	0		
DP	$A_1(0)$	$a_{i,1}(0)$	$a_{i,2}(0)$			$a_{i,3}(0)$		$a_{i,4}(0)$	$a_{i,5}(0)$	$a_{i,6}(0)$			$a_{i,7}(0)$		
	Before hashing	1		3		2		1	0	1			2		
	After hashing	1		1		0		0	0	1			0		

5.1. Quantum population initialization

In QPSO, the quantum particle (QP) representation is designed to explicitly address the offloading decision of QTsks to QNdS. A QP is denoted as $\vec{Q}_i(t) = \{q_{i,1}(t), \dots, q_{i,D}(t)\}$, where D is the dimension of the QP. Each component $q_{i,m}(t)$ is a qubit and represented by a two-dimensional vector: $q_{i,m}(t) = \begin{bmatrix} \alpha_{i,m}(t) \\ \beta_{i,m}(t) \end{bmatrix}$.

The amplitudes $\alpha_{i,m}(t)$ and $\beta_{i,m}(t)$ are initialized as $\alpha_{i,m}(t) = \frac{x}{\sqrt{x^2 + y^2}}$ and $\beta_{i,m}(t) = \frac{y}{\sqrt{x^2 + y^2}}$, where x and y are random real numbers drawn from the interval $(-\infty, \infty)$, as given in Algorithm 1. An initial quantum population $Q_{POP}(t) = \{\vec{Q}_1(t), \dots, \vec{Q}_{P_{size}}(t)\}$ of P_{size} QPs is generated randomly (initially, $t = 0$). Each QP encodes a complete solution to the QTOP. For example, if QTsk ϕ_g can be offloaded to o_g possible QNdS (see Section 3.2), then $QB_g = \lceil \log_2(o_g) \rceil$ qubits are required to represent QB_g . Note, when $o_g = 1$, then QB_g is taken as 1. The first QB_1 qubits are used to represent the offloading decision of ϕ_1 . Similarly, the next QB_2 qubits are used for ϕ_2 and so on. Hence, the dimension of a QP is given

by: $D = \sum_{g=1}^M QB_g$, where M is the total number of QTsks. Algorithm 1 highlights the entire process.

Illustration 5.1. Consider seven QTsks ($M = 7$) are generated. There are 3 QNdS, $\{n_1, n_2, n_3\}$, i.e., $N = 3$. The scale of QNdS and the number of qubits required to execute 7 QTsks are shown in Table 4. The set of options can be represented as $\mathcal{O} = \{3, 2, 3, 1, 1, 2, 2\}$. It can be said that the first QTsk has the option of offloading in QNd n_1, n_2 , and n_3 . The number of quantum bits (QB) required to represent a QTsk with 3 options is computed as $QB = \lceil \log_2 3 \rceil = 2$. Similarly, for QTsk instances with 2 and 1 options, the required number of QB qubits is 2 and 1, respectively. Hence, the total dimension required to represent the QP is $D = 2 + 2 + 2 + 1 + 1 + 2 + 2 = 12$. Table 4 shows the representation of the discussed process.

Lemma 5.1. Time complexity of $Initialize(P_{size}, D)$ is $O(P_{size} \times D)$.

Proof. A QP is created in $O(D)$ times (line 4–11, Algorithm 1). A Q_{POP} with P_{size} QP is created in $O(P_{size} \times D)$ time. $\square$

Algorithm 1 Initialize(P_{size}, D).**Input:** Dimension of the QP: D and Population size: P_{size} . **Output:** $Q_{POP}(t)$.

```

1: Initialize  $\bar{Q}_{POP}(t) = \{\}$ 
2: Initialize  $\bar{Q}_i(t) = \{\}, \forall i, 1 \leq i \leq P_{size}$ 
3: for  $i = 1$  to  $P_{size}$ 
4:   for  $m = 1$  to  $D$ 
5:      $x = rand[-\infty, \infty]$ 
6:      $y = rand[-\infty, \infty]$ 
7:      $\alpha_{i,m}(t) = \frac{x}{\sqrt{x^2 + y^2}}$ 
8:      $\beta_{i,m}(t) = \frac{y}{\sqrt{x^2 + y^2}}$ 
9:      $q_{i,m}(t) = \begin{bmatrix} \alpha_{i,m}(t) \\ \beta_{i,m}(t) \end{bmatrix}$ 
10:     $\bar{Q}_i(t) = \bar{Q}_i(t) \cup q_{i,m}(t)$ 
11:   end for
12:    $Q_{POP}(t) = Q_{POP}(t) \cup \bar{Q}_i(t)$ 
13: end for
14: return  $Q_{POP}(t)$  /*Returns the quantum population*/

```

5.2. Measuring quantum population

A QP $\bar{Q}_i(t)$ is measured to produce a binary particle (BP) $B_i(t) = \{b_{i,1}(t), b_{i,2}(t), \dots, b_{i,D}(t)\}$, $b_{i,m} \in \{0, 1\}$. **Algorithm 2** reflects the process.

Illustration 5.2. A quantum particle (QP) is initialized following Illustration 5.1 and presented in **Table 4**. To evaluate each qubit $q_{i,m}(t)$, the corresponding binary value $b_{i,m}(t)$ is obtained through measurement using **Algorithm 2**. Applying this measurement approach across the entire QP $\bar{Q}_1(0)$, a BP $B_1(0)$ is generated, as shown in **Table 4**.

Algorithm 2 Measure($\bar{Q}_i(t)$).

```

Input: QP,  $\bar{Q}_i(t) = \begin{bmatrix} \alpha_{i,1}(t) & \dots & \alpha_{i,m}(t) & \dots & \alpha_{i,D}(t) \\ \beta_{i,1}(t) & \dots & \beta_{i,m}(t) & \dots & \beta_{i,D}(t) \end{bmatrix}$ 
Output: BP,  $B_i(t) = \{b_{i,1}(t), \dots, b_{i,m}(t), \dots, b_{i,D}(t)\}$ .
1: Initialize  $B_i(t) = \{\}$ 
2: for  $m = 1$  to  $D$  do
3:   if  $rand[0, 1] \leq |\alpha_{i,m}|^2$  then
4:      $b_{i,m}(t) = 0$ 
5:   else
6:      $b_{i,m}(t) = 1$ 
7:   end if
8:    $B_i(t) = B_i(t) \cup b_{i,m}(t)$ 
9: end for
10: return  $B_i(t)$  /*Returns the corresponding BP*/

```

Lemma 5.2. The Measure($\bar{Q}_i(t)$) operation has a time complexity of $O(D)$.

Proof. Every qubit is observed to obtain its equivalent binary bit as specified in **Algorithm 2**. Given that the dimension of the QP is D , the time complexity of this process is $O(D)$. $\square$

5.3. Decoding of binary population

A binary particle (BP) $B_i(t)$ of dimension D is divided into M fragments, where $D = QB_1 + QB_2 + \dots + QB_M$. Each fragment of length QB_g corresponds to a QTask ϕ_g and is decoded from binary to its decimal equivalent to form the decoded particle (DP) $A_i(t) = \{a_{i,1}(t), \dots, a_{i,M}(t)\}$. Specifically, $a_{i,1}(t)$ is obtained by decoding the first QB_1 bits, i.e., $[b_{i,1}(t), \dots, b_{i,QB_1}(t)]_2$. Similarly, $a_{i,2}(t)$ is derived from the next QB_2 bits, i.e., $[b_{i,QB_1+1}(t), \dots, b_{i,QB_1+QB_2}(t)]_2$, and in general, $a_{i,g}(t)$ is the decimal value of the binary string $[b_{i,QB_{g-1}+1}(t), \dots, b_{i,QB_{g-1}+QB_g}(t)]_2$.

Each $a_{i,g}(t) \in [0, 2^{QB_g} - 1]$ represents the index of the QNd assigned to task ϕ_g . If the decoded value exceeds the available number of QNds

Algorithm 3 Decode($B_i(t)$).**Input:** BP, $B_i(t) = \{b_{i,1}(t), b_{i,2}(t), \dots, b_{i,D}(t)\}$ **Output:** DP, $A_i(t) = \{a_{i,1}(t), \dots, a_{i,g}(t), \dots, a_{i,M}(t)\}$

```

1: Initialize  $A_i(t) = \{a_{i,1}(t), a_{i,2}(t), \dots, a_{i,M}(t)\}$ 
2: Initialize  $g = 1$ 
3: Initialize  $Base = 2^{QB_g-1}$ ,  $d = 1$ 
4: Initialize  $a_{i,g}(t) = 0$ ,  $BitCount = 0$ 
5: while  $m \leq D$  do
6:    $a_{i,g}(t) = a_{i,g}(t) + b_{i,m}(t) \times Base$ 
7:    $Base = Base/2$ 
8:    $BitCount = BitCount + 1$ 
9:   if ( $BitCount == QB_g$ ) then
10:    if  $a_{i,g}(t) > (o_g - 1)$  then
11:       $a_{i,g}(t) = a_{i,g}(t) \bmod o_g$  /*Hashing*/
12:    end if
13:     $A_i(t) = A_i(t) \cup a_{i,g}(t)$ 
14:     $Base = 2^{QB_g-1}$ 
15:     $g = g + 1$ 
16:     $a_{i,g}(t) = 0$ ,  $BitCount = 0$ 
17:  end if
18:   $m = m + 1$ 
19: end while
20: return  $A_i(t)$  /*Returns the corresponding DP*/

```

for ϕ_g (i.e., $a_{i,g}(t) > o_g - 1$), a linear hashing technique is applied to map it into the valid range $[0, o_g - 1]$, as described in **Algorithm 3**.

Illustration 5.3. Considering Illustrations 5.1 and 5.2. Since $QB_2 = \lceil \log_2(o_2) \rceil = 2$, every two-bit binary string is decoded into its corresponding decimal format. For instance, $a_{i,2}(0) = 3$ is decoded from the binary string $[b_{i,3}(0), b_{i,4}(0)] = [11]_2$. Similarly, $a_{i,2}(0) = 2$ is decoded from $[b_{i,3}(0), b_{i,4}(0)] = [10]_2$. Given $o_2 = 2$, the valid set of QNds is $\{0, 1, 2\}$. Consequently, $a_{i,2}(0) = 3$ is hashed linearly to $a_{i,2}(0) = 1$, as displayed in line 11 of **Algorithm 3**. Therefore, ϕ_2 is offloaded to QNd n_1 . The complete DP, both before hashing and after hashing, is summarized in **Table 4**.

Lemma 5.3. Decode($B_i(t)$) has a time complexity of $O(D)$.

Proof. Line no. 5–19 from **Algorithm 3** loops for D times. Hashing at line no. 11 needs $O(1)$ time. Thus, $O(D)$ is the time complexity. $\square$

5.4. Evaluation of fitness

The fitness of a decoded particle (DP) $A_i(t)$ represents the quality of the solution. Since the execution of QTsk is prioritized based on their associated urgency levels, the DP $A_i(t)$ is accordingly sorted. Initially, the QTsk are arranged in ascending order of urgency, that is, $U_{\pi(1)} \leq U_{\pi(2)} \leq \dots \leq U_{\pi(M)}$, where π denotes a permutation of the index set $\{1, 2, \dots, M\}$. Consequently, the sorted DP list is represented as $UA = \{a_{i,\pi(1)}(t), a_{i,\pi(2)}(t), \dots, a_{i,\pi(g)}(t), \dots, a_{i,\pi(M)}(t)\}$. The QTsk are assigned one by one based on the UA . The objectives are described below:

- **Objective 1 (Minimization of delay):** The total delay, $\delta^t = \frac{\delta^t}{\delta^m}$, is calculated as discussed in **Section 5**. It is important to note that the maximum value of δ^t is 1. According to the constraint specified in **Eq. (26)**, every QTsk is subject to a time deadline. A penalty is imposed for violating this deadline, as follows:

$$\delta^t = \begin{cases} 1, & \text{if } (\exists \phi_g, s.t., \delta_g > \phi_g^T) \\ \delta^t, & \text{Otherwise.} \end{cases} \quad (33)$$

- **Objective 2 (Minimization of energy):** The total energy (e^t) consumption is calculated as $e^t = \frac{e^t}{e^m}$, is calculated as discussed in **Section 5**.
- **Objective 3 (Minimization of load balance):** The total load is calculated as $LB = \frac{\text{Var}(\sigma)}{\sigma^m}$ as discussed in **Section 5**.

- **Objective 4 (Minimization of price):** The total price is calculated as $\mathbb{P}^t = \frac{\mathbb{P}^t}{\mathbb{P}^m}$ as discussed in Section 5.

After penalty calculation, the δ^t is updated. The entire fitness is calculated as:

$$Fitness = \omega_1 \cdot e^t + \omega_2 \cdot \delta^t + \omega_3 \cdot LB + \omega_4 \cdot \mathbb{P}^t \quad (34)$$

The objective is to minimize *Fitness*. Algorithm 4 is used to calculate the fitness of a DP.

Lemma 5.4. The time complexity of $Fitness(A_i(t))$ is $O(M \log M)$.

Proof. The DP is first sorted based on task urgency (Line 5, Algorithm 4). It requires $O(M \log M)$ time. The operations in Lines 7–15 are executed in $O(M)$ time, while those in Lines 16–18 have constant time complexity, i.e., $O(1)$. Line 19 also runs in $O(M)$ time, and Line 20 in $O(N)$ time. Finally, the operations in Lines 21–24 again have constant time complexity, $O(1)$. Hence, the overall time complexity of the fitness evaluation procedure is $O(M \log M)$. $\square$

Algorithm 4 : $Fitness(A_i(t))$.

Input: $A_i(t) = \{a_{i,1}(t), \dots, a_{i,g}(t), \dots, a_{i,M}(t)\}$. **Output:** Fitness of $A_i(t)$.

- 1: Initialize $flag_1 = 0, flag_2 = 0, e^t = \delta^t = \mathbb{P}^t = 0$.
- 2: Initialize $\omega_i, \forall i \in [1, 4]$.
- 3: Calculate $\delta^m, e^m, \mathbb{P}^m, \sigma^m$ // Eqs. (8), (11), (16), (13)
- 4: Calculate $U_g, \forall g \in \{1, M\}$. // Using Eq. (17).
- 5: Sort $A_i(t)$ as $U A_i(t) = \{a_{i,\pi(1)}(t), a_{i,\pi(2)}(t), \dots, a_{i,\pi(M)}(t)\}$ such that $U_{\pi(1)} \leq U_{\pi(2)} \leq \dots \leq U_{\pi(M)}$
- 6: Initialize $\delta_g = 0, \forall g \in [1, M], \sigma_j = 0, \forall j \in [1, N]$
- 7: **for** $g = 1$ to M // One by one from sorted list.
- 8: $j = a_{i,\pi(g)}(t)$
- 9: $\delta_g = \delta_{g,j}^c + \delta_g^w$ // Using Eq. (5).
- 10: **if** $(\delta_g > \phi_g^T)$ **then** $flag_1++$ // Using Eq. (33).
- 11: $e^t = e^t + (\delta_{g,j}^c \times \phi_g^q \times p_j)$ // Using Eq. (10)
- 12: $\mathbb{P}^t = \mathbb{P}^t + (\rho_j \times \delta_{g,j}^c)$ // Using Eq. (15)
- 13: **if** $((\rho_j \times \delta_{g,j}^c) > \phi_g^P)$ **then** $flag_2++$ // Eq. (27).
- 14: $\sigma_j = \sigma_j + \phi_g^d$
- 15: **end for**
- 16: $\delta^t = \frac{1}{\delta^m} \max\{\delta_g, \forall g \in [1, M]\}$
- 17: $e^t = \frac{e^t}{e^m}$
- 18: $\mathbb{P}^t = \frac{\mathbb{P}^t}{\mathbb{P}^m}$
- 19: $\bar{\sigma} = \frac{1}{N} \sum_{g=1}^M \phi_g^d$
- 20: $Var(\sigma) = \frac{1}{N} \sum_{j=1}^N (\sigma_j - \bar{\sigma})^2$ // Using Eq. (12)
- 21: $LB = \frac{Var(\sigma)}{\sigma^m}$
- 22: **if** $(flag_1 > 0)$ **then** $\delta^t = 1$. // Penalty for delay.
- 23: **if** $(flag_2 > 0)$ **then** $\mathbb{P}^t = 1$. // Penalty for price.
- 24: $Fitness = \omega_1 \times \delta^t + \omega_2 \times e^t + \omega_3 \times LB + \omega_4 \times \mathbb{P}^t$
- 25: **return** *Fitness* /*Returns fitness for the DP.*/

5.5. Updation of quantum particles

Each QP $\vec{Q}_i(t)$ can also be represented using angular form $\vec{\Theta}_i(t)$ as:

$$\vec{\Theta}_i(t) = \{\theta_{i,1}(t), \dots, \theta_{i,m}(t), \dots, \theta_{i,D}(t)\} \quad (35)$$

where each angle is computed as $\theta_{i,m}(t) = \arctan\left(\frac{\beta_{i,m}(t)}{\alpha_{i,m}(t)}\right)$. Then the qubits are updated using the following two phases:

- Updating the angles $\Delta\vec{\Theta}_i(t)$ using the equation:

$$\begin{aligned} \Delta\vec{\Theta}_i(t+1) = & w \cdot \Delta\vec{\Theta}_i(t) + \kappa_1 \cdot r_1 \cdot \{\vec{\Theta}_i^p(t) - \vec{\Theta}_i(t)\} \\ & + \kappa_2 \cdot r_2 \cdot \{\vec{\Theta}^g(t) - \vec{\Theta}_i(t)\} \end{aligned} \quad (36)$$

Here, w is the inertia weight, κ_1 and κ_2 are acceleration constants, and r_1, r_2 are random values in $[0, 1]$. $\vec{\Theta}_i^p(t)$ and $\vec{\Theta}^g(t)$ represent the personal best and global best angles, respectively.

Table 5

Time complexity analysis of the various QPSO stages (Lemma 5.5).

Module	Complexity	Frequency	Total complexity
Initialize(P_{size}, D)	$O(P_{size} \times D)$	1	$O(P_{size} \times D)$
Measure($\vec{Q}_i(t)$)	$O(D)$	$O(P_{size} \times MAX)$	$O(P_{size} \times MAX \times D)$
Decode($B_i(t)$)	$O(D)$	$O(P_{size} \times MAX)$	$O(P_{size} \times MAX \times D)$
Fitness($A_i(t)$)	$O(M \log M)$	$O(P_{size} \times MAX)$	$O(P_{size} \times MAX \times M \log M)$

- Updating the qubits' angular positions:

$$\begin{bmatrix} \alpha_{i,m}(t+1) \\ \beta_{i,m}(t+1) \end{bmatrix} = R(\Delta\theta_{i,m}(t)) \cdot \begin{bmatrix} \alpha_{i,m}(t) \\ \beta_{i,m}(t) \end{bmatrix} \quad (37)$$

where,

$$R(\Delta\theta_{i,m}(t)) = \begin{bmatrix} \cos(\Delta\theta_{i,m}(t)) & -\sin(\Delta\theta_{i,m}(t)) \\ \sin(\Delta\theta_{i,m}(t)) & \cos(\Delta\theta_{i,m}(t)) \end{bmatrix}$$

is the 2×2 quantum rotation gate [34].

Remark 5.1. In QPSO, each QP is represented using qubits, enabling a probabilistic encoding of potential solutions. In classical PSO, each particle component is a deterministic real number. The quantum angles in QPSO are updated using Eq. (36). The qubits of a QP are updated using the quantum rotation gate as in Eq. (37). A QP of dimension D inherently encodes a superposition of 2^D possible states simultaneously. In contrast, a classical particle of the same dimension represents only a single candidate solution at any given time. Moreover, classical PSO relies on velocity and position updates governed by Eqs. (31) and (32), which follow a fixed trajectory influenced by cognitive and social components. By using QPs such as superposition and qubit representation, QPSO inherently promotes richer exploration and more effective exploitation of the solution space. Thus, the use of QPSO provides a more powerful and flexible optimization compared to classical PSO.

The fitness of each particle is evaluated, and the corresponding personal best ($\vec{\Theta}_i^p$) and the global best ($\vec{\Theta}^g$) particles are updated iteratively. The process continues until the terminating condition is met. The final solution is the global best particle, which has the best fitness in the population. The QPSO algorithm, as detailed in Algorithm 4, integrates the complete QPSO process outlined in Sections 5.1–5.5.

Lemma 5.5. The time complexity of $QPSO()$ is $O(P_{size} \times MAX \times D)$.

Proof. The initialization phase has a time complexity of $O(P_{size} \times D)$, as stated in Lemma 5.1. The measurement phase requires $O(D)$ time for each QP (Lemma 5.2), and the decoding step also takes $O(D)$ time (Lemma 5.3). Task partitioning is performed in constant time, i.e., $O(1)$. According to Lemma 5.4, the fitness evaluation is computed in $O(M \log M)$ time. Among these operations, the initialization (Line 2, Algorithm 5) is executed only once, whereas all other steps are iterated $O(P_{size} \times MAX)$ times. Therefore, the overall time complexity of the QPSO algorithm is $O(P_{size} \times MAX \times D)$, as summarized in Table 5. $\square$

6. Results and analysis

6.1. Simulation scenario

Experiments are conducted on a computer with a 64-bit version of Windows 11 running at 1.30 GHz, a 12th Generation Intel® Core™ i5-1235U CPU, and 16 GB of RAM. Python 3.9 and MATLAB online programming are used for the simulation.

By using the proposed fitness function and encoding technique, the models based on QTO are developed using various algorithms, including GA, PSO, GSA, and differential evolution (DE), for comparison with the proposed QPSO. Moreover, the QPSO is compared with the existing DRL algorithm [2]. A detailed comparison is made between different price models using QPSO (as discussed in Section 6.1.1). Subsequently, the proposed models are further compared with those of GA, PSO, GSA, DE,

Table 6
Values and parameters (Section 6.1).

Notation	Values	Notation	Values	Notation	Values	Notation	Values
P_{size}	50	n_j^q	$2.2 \times 10^7 - 9 \times 10^7$	ϕ_g^q	$2.2 \times 10^7 - 9 \times 10^7$	ϕ_g^T	5000 seconds
ϕ_g^d	$1.5 \times 10^4 - 2.9 \times 10^4$	n_j^s	700–1500	p_j	$2 \times 10^4 W - 9 \times 10^7 W$	MAX	100
ϕ_g^s	1000 - 1050	M	100 - 500	ρ_j	\$1 - \$3 per second		
ϕ_g^q	$2.2 \times 10^7 - 9 \times 10^7$	N	3 - 20	$\omega_1, \omega_2, \omega_3, \omega_4$	0.25		

Algorithm 5 QPSO.

Input: Initial population size: P_{size} , maximum iteration: MAX , dimension: D

Output: Global best DP: A^g

```

1: Initialize  $t = 0$ 
2: Initialize  $Q_{POP}(t) = \text{Initialize}(P_{size}, D)$  // Algorithm 1
3: for  $i = 1$  to  $P_{size}$  do
4:    $B_i(t) = \text{Measure}(\vec{Q}_i(t))$  // Algorithm 2
5:    $A_i(t) = \text{Decode}(B_i(t))$  // Algorithm 3
6:   Compute  $\vec{\Theta}_i(t)$  // Using Eq. (35)
7:    $\vec{\Theta}_i^p = \vec{\Theta}_i(t)$  // Update personal best
8:    $f_i = \text{Fitness}(A_i(t))$  // Algorithm 4
9: end for
10:  $\vec{\Theta}^g = \vec{\Theta}_k(t)$  such that  $f_k = \min\{f_i, \forall i \in [1, P_{size}]\}$ 
11:  $A^g = A_k(t)$  such that  $f_k = \min\{f_i, \forall i \in [1, P_{size}]\}$ 
12: while  $t \leq MAX$  do
13:   for  $i = 1$  to  $P_{size}$  do
14:     Compute  $\Delta\vec{\Theta}_i(t+1)$  // Eq. (36)
15:     Update  $\begin{bmatrix} \alpha_{i,m}(t+1) \\ \beta_{i,m}(t+1) \end{bmatrix}, \forall m \in [1, D]$  // Eq. (37)
16:      $B_i(t+1) = \text{Measure}(\vec{Q}_i(t+1))$  // Algorithm 2
17:      $A_i(t+1) = \text{Decode}(B_i(t+1))$  // Algorithm 3
18:     if  $\text{Fitness}(A_i(t+1)) < f_i$  then
19:        $f_i = \text{Fitness}(A_i(t+1))$ 
20:        $\vec{\Theta}_i^p = \vec{\Theta}_i(t+1)$  // Update personal best
21:       if  $f_i < \text{Fitness}(A^g)$  then
22:          $\vec{\Theta}^g = \vec{\Theta}_i(t+1)$  // Update global best
23:          $A^g = A_i(t+1)$  // Update global best
24:       end if
25:     end if
26:   end for
27:    $t = t + 1$ 
28: end while
29: return  $A^g$  //Global best DP.*/

```

Table 7

Price chart in \$ for proposed cost (PC), per shot (PS) [6], and per task (PT) [6] (Section 6.1.1).

	n_1	n_2	n_3	n_4	n_5	n_6	n_7	n_8
PC	1	1	1.6	2.7	1	1	1	1
PT	1	1	1.6	2.7	1	1	1	1
PS	1.3	1.3	1.3	1.3	1.3	1.3	1.3	1.3

and DRL [2] based on scenario 1 (Section 6.1.2) and scenario 2 (Section 6.1.3). The experiments conducted on QPSO for scenarios 1 and 2 cover a range of tasks varying from 100 to 500. In this study, the simulation parameters are adopted from industry standards and prior research in the field, as detailed in Tables 6–8 [2,5,6,13–15,30,34]. These parameter settings represent standard, realistic, and widely accepted practices for evaluating QTO algorithms.

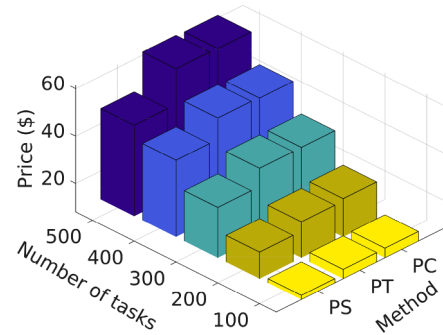
6.1.1. Cost model comparison

In this analysis, the proposed cost (PC) model is compared with the per task (PT) model [6] and the per shot (PS) model [6]. For the experiment, a total of eight QNds ($n_1, \dots, n_8$) is considered. In the PC model,

Table 8

Price comparison in \$ between PC, PS [6], and PT [6], and PT [6] in 10^6 (Section 6.1.1).

	100	200	300	400	500
PC	1.20	2.41	3.70	4.89	6.10
PT	1.19	2.31	3.69	4.93	6.13
PS	0.97	1.91	2.92	4.03	4.63

**Fig. 5.** Cost model comparison in 10^5 (PS: Per shot, PT: Per task, PC: Proposed cost) (Section 6.1.1).

the price is charged per second. Whereas for the PS model, the price is charged per shot. For the PT model, the price is charged per second and is the same for all QNds. Table 7 reflects the pricing chart for these models. Moreover, the comparison between PC, PS, and PT models using QPSO is done for 100, 200, 300, 400, and 500 QTsks.

As shown in Fig. 5 and Table 8, the PT model results in lower charges. However, this approach is ineffective, as it fails to account for the varying computational costs associated with different QNd types. A key limitation of the PT model is that it applies a uniform price per second without considering the specific type of QNd used. Pricing should be determined based on the type of QNd to ensure fairness and efficiency. Similarly, the PS model results in lower charges (Fig. 5 and Table 8). A key limitation of the PS model is its inability to consider the actual runtime of QTsks and charge them accordingly. This results in an unfair pricing strategy making the PS model less effective compared to the PC strategy, which ensures a more balanced and justified pricing approach.

6.1.2. Scenario-1

Scenario 1 has considered five QNds for the simulation. The performance comparison is carried out across GA, PSO, GSA, DE, and DRL, focusing on key optimization objectives. Figs. 6(a)–(d), and 8(a) illustrate the comparative results with respect to delay, energy consumption, load balancing, price, and overall fitness, respectively. QPSO exhibited superior performance in minimizing both price and load imbalance. In terms of delay reduction, both GA and QPSO achieved the most favorable results. Additionally, QPSO demonstrated improved energy efficiency compared to PSO, GA, GSA, and DE. DRL performed well in load balancing and energy consumption. This is a result of DRL's improved QTsk distribution among the QNds. QPSO performed better than DRL [2] in the case of delay and price. Overall, QPSO has outperformed the existing algorithms across multiple objectives.

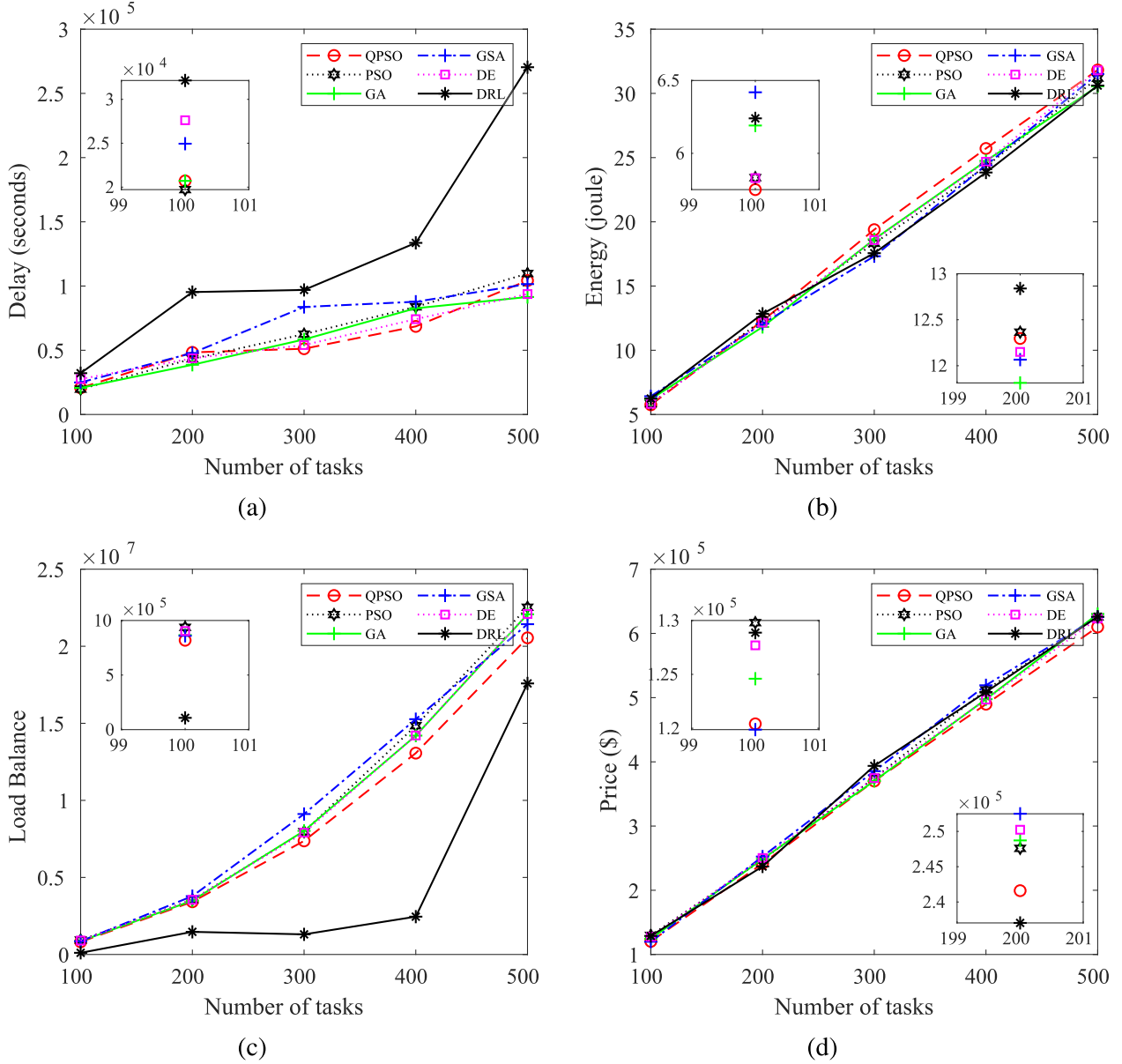


Fig. 6. Comparison on the Scenario - 1 (Section 6.1.2), in terms of (a) Delay (b) Energy (c) Load balance, and (d) Price.

6.1.3. Scenario-2

In this scenario, twenty QNds are considered. Similar to scenario 1 (Section 6.1.2), a comparative analysis of delay, energy, load balance, price, and overall fitness is conducted between the proposed QPSO and other algorithms. QPSO effectively performed better than others while minimizing price (Fig. 7(d)). In terms of minimizing energy consumption (Fig. 7(b)), QPSO outperformed other algorithms on multiple occasions. For delay (Fig. 7(a)) and load balance (Fig. 7(c)), QPSO performed better than other algorithms. In contrast to scenario 1, QPSO demonstrates improved performance across delay, energy, price, and load balance metrics as the number of QNds increases. Thus, QPSO demonstrated superior performance in minimizing overall fitness (Fig. 8(b)) by effectively making a trade-off between all the objectives.

In both Scenario 1 and Scenario 2, the proposed QPSO algorithm demonstrated consistently strong performance across multiple evaluation metrics when compared to GA, GSA, PSO, DE, and DRL. Regarding delay, QPSO achieved significant reductions, outperforming the base-

line algorithms by up to 61.43% in Scenario 1 and 23.53% in Scenario 2. In terms of energy efficiency, QPSO provided a performance improvement of approximately 10% in both scenarios, indicating its effectiveness in minimizing energy consumption during QTO. For the cost metric, QPSO exhibited a favorable trade-off between service quality and price, achieving up to a 7.5% improvement over competing approaches. Furthermore, with respect to load balancing, QPSO demonstrated superior distribution of computational tasks, with improvements of up to 19.14% in Scenario 1 and 33.66% in Scenario 2. These results collectively underscore QPSO's robustness and adaptability in optimizing multiple, often conflicting, objectives under varying workloads and system configurations. The consistent performance of QPSO across delay, energy, cost, and load balancing metrics validates its effectiveness in addressing the complexities of the quantum task offloading problem.

Classical metaheuristics algorithms struggle with fine-grained adjustments, resulting in higher delay, energy consumption, cost, and load imbalance, especially as task size and the number of QNds increase. In contrast, QPSO leverages quantum behavior, enabling a superior

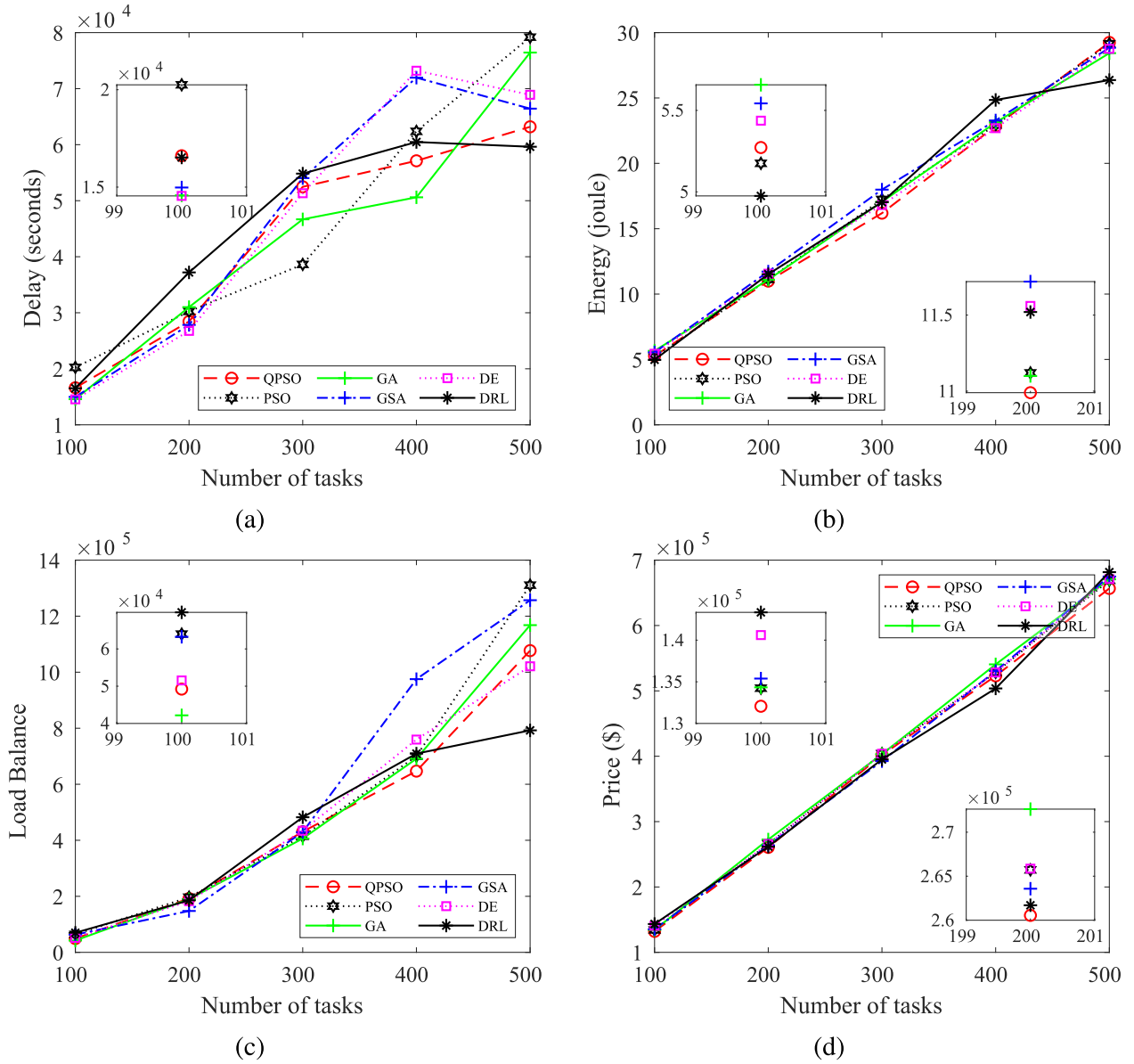


Fig. 7. Comparison on the Scenario-2 (Section 6.1.3), in terms of (a) Delay, (b) Energy, (c) Load balance, and (d) Price.

convergence rate along with enhanced exploration and exploitation capabilities.

6.2. Analysis of variance (ANOVA)

ANOVA analyses the result's significance [15].

As per Table 9, H_0 is rejected since $p\text{-value} < \alpha$. It's thought that the averages of various groups are not the same. F -statistic, 18.363, exceeds 95 % limit, or $[-\infty : 2.579]$ acceptability area.

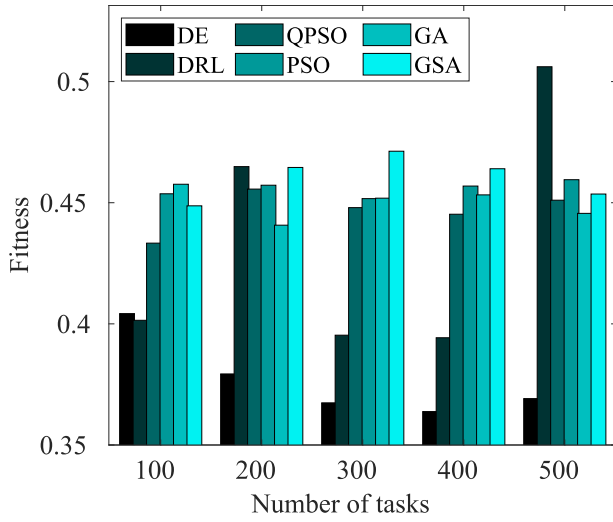
6.3. Exploration and exploitation

Exploration and exploitation capabilities of the proposed QPSO are evaluated by first determining the population's divergence. Subsequently, the percentage of exploration and exploitation is computed, following the approach outlined in Morales-Castañeda et al. [35]. The population diversity (Div) is calculated as:

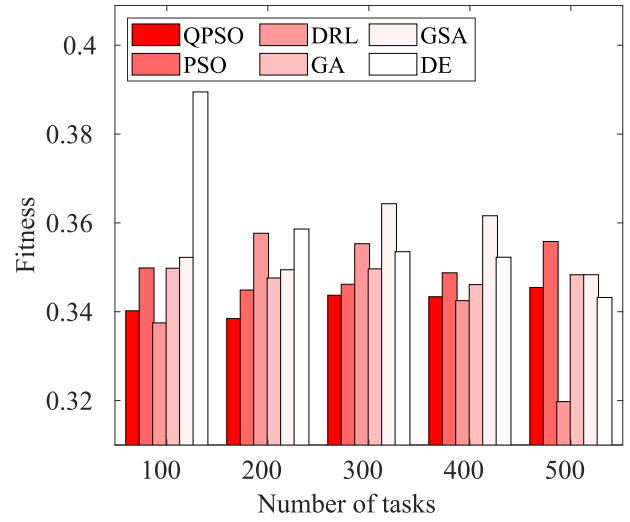
$$Div = \frac{1}{P_{size}} \sum_{g=1}^M Div_g \quad (38)$$

Table 9
ANOVA performed on overall fitness with hundred tasks on scenario-2 (Section 6.2).

Source of variations	Sum of Square	Degree of Freedom	Mean Square	F-statistic	p-value	F-critical
Between Groups	0.01772	4	0.004429	18.3637	5.2×10^{-9}	2.579
Within Groups	0.01085	45	0.0002412			
Total	0.02857	49	0.0005831			

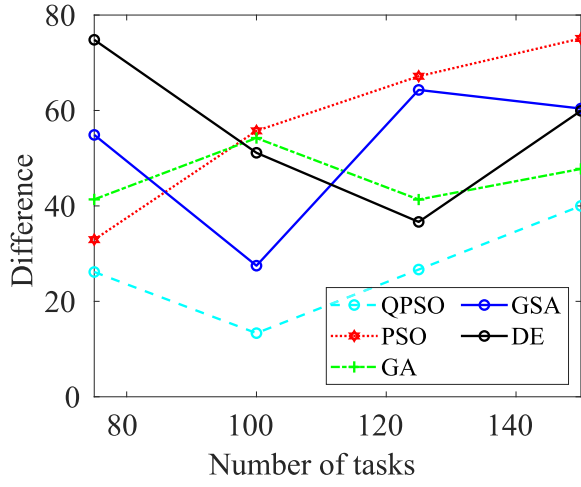


(a)

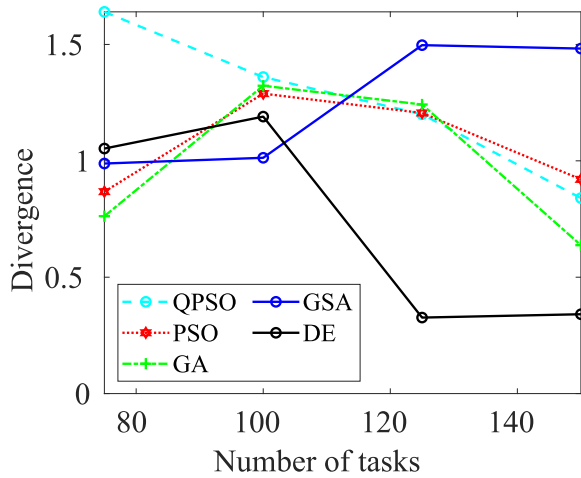


(b)

Fig. 8. Fitness comparison (Sections 6.1.2–6.1.3), (a) scenario 1, (b) scenario 2.



(a)



(b)

Fig. 9. Comparison (Section 6.3), in terms of (a) Difference between exploration and exploitation, and (b) Divergence.

Table 10

Exploration values (Section 6.3).

	75	100	125	150
QPSO	63.07	56.67	63.33	70
PSO	66.46	77.86	83.58	87.54
GA	70.68	77.11	70.67	73.87
GSA	77.44	63.73	82.15	80.20
DE	87.40	75.56	68.32	79.96

Table 11

Exploitation values (Section 6.3).

	75	100	125	150
QPSO	36.92	43.33	36.66	30
PSO	33.53	22.13	16.41	12.45
GA	29.31	22.88	29.33	26.12
GSA	22.55	36.26	17.84	19.79
DE	12.59	24.43	31.67	20.03

The dimension-wise diversity for the g th dimension is defined as $|\text{median}(a_g) - a_{i,g}|$, where $\text{median}(a_g)$ represents the median value of dimension g across the entire population, and $a_{i,g}$ denotes the g th dimension value of the i th solution within the population.

Here, P_{size} signifies the total population size, while M represents the number of dimensions. The percentages of exploration (EPL) and exploitation (EPT) at each iteration are determined using the following equations:

$$EPL = \left(\frac{Div}{Div_{\max}} \right) \times 100 \% \quad (39)$$

$$EPT = \left(\frac{Div - Div_{\max}}{Div_{\max}} \right) \times 100 \% \quad (40)$$

where $Div_{\max}$ denotes the highest diversity value observed across the entire population. Unlike [35], in this research if $Div_{\max} = 0$ then $EPL = 0$, $EPT = 0$. In Scenario 2, a population with $P_{size} = 10$ is considered to assess exploration and exploitation across varying QTsk sizes: 75, 100, 125, and 150. The exploration and exploitation values obtained from these experiments are presented in Table 10 and Table 11. Fig. 9(a) shows that the difference between exploration and exploitation is the least for QPSO among all the compared algorithms. Moreover, Fig. 9(b) proves that QPSO exhibits the balanced divergence capability.

7. Conclusion

This study presents QPSO for QTO in multi-application user cloud computing systems. Its goal is to reduce load balancing, cost, delay, and energy usage. The proposed cost model is analyzed in comparison with existing models, highlighting its effectiveness. When combined with QPSO, the suggested QTO model performs better overall than PSO, GA, GSA, DE, and DRL.

The proposed QTO model is applicable across industrial IoT (IIoT) and smart manufacturing, research, defense, and battlefield surveillance. While the proposed QTO model performs effectively in its current form, integrating cold start mitigation policies, quantum error correction, and quantum coherence techniques in the future could enhance the QCC's robustness and overall performance in practical applications.

CRedit authorship contribution statement

Santanu Ghosh: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization; **Pratyay Kuila:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Formal analysis, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] Quantum Computing Applications: Evolution & Potential Impact — tcs.com, 2024, (<https://www.tcs.com/what-we-do/pace-innovation/white-paper/quantum-computing-technology-applications>). [Accessed 27-12-2024].
- [2] H.T. Nguyen, M. Usman, R. Buyya, DRLQ: a deep reinforcement learning-based task placement for quantum cloud computing, in: 2024 IEEE 17th International Conference on Cloud Computing (CLOUD), Institute of Electrical and Electronics Engineers (IEEE), Los Alamitos, CA, USA, 2024, pp. 475–481. <https://doi.org/10.1109/CLOUD62652.2024.00060>
- [3] M. Golec, E.S. Hatay, M. Golec, M. Uyar, M. Golec, S.S. Gill, Quantum cloud computing: trends and challenges, J. Econ. Technol. 2 (2024) 190–199. <https://doi.org/10.1016/j.ject.2024.05.001>
- [4] A.A. Zahir, S.M. Mohd, M.S. Mohd Ilias, B. Idrus, H. Zainuddin, N. Mohamad Jan, M.R. Wahiddin, Quantum computing in the cloud—a systematic literature review, Int. J. Electr. Comput. Eng. Syst. 15 (2) (2024) 185–200. <https://doi.org/10.32985/ijeces.15.2.7>
- [5] I. Cloud, IBM Cloud Docs — cloud.ibm.com, 2024, (<https://cloud.ibm.com/docs/quantum-computing?topic=quantum-computing-plans>). [Accessed 22-12-2024].
- [6] AWS, Quantum computer and simulator—amazon braket pricing—AWS—aws.amazon.com, 2024, (<https://aws.amazon.com/braket/pricing/>). [Accessed 22-12-2024].
- [7] H.T. Nguyen, M. Usman, R. Buyya, QFaaS: a serverless function-as-a-service framework for quantum computing, Future Gener. Comput. Syst. 154 (2024) 281–300. <https://doi.org/10.1016/j.future.2024.01.018>
- [8] F. Leymann, S. Barzen, M. Falkenthal, D. Vietz, B. Weder, K. Wild, Quantum in the cloud: application potentials and research opportunities (2020). <https://doi.org/10.5220/0009819800090024>
- [9] S. Jin, N. Liu, Y. Yu, Time complexity analysis of quantum algorithms via linear representations for nonlinear ordinary and partial differential equations, J. Comput. Phys. 487 (2023) 112149.
- [10] S.S. Gill, A. Kumar, H. Singh, M. Singh, K. Kaur, M. Usman, R. Buyya, Quantum computing: a taxonomy, systematic review and future directions, insSoftware 52 (1) (2022) 66–114. <https://doi.org/10.1002/spe.3039>
- [11] L. Aggarwal, S. Sachdeva, P. Goswami, Quantum healthcare computing using precision based granular approach, Appl. Soft Comput. 144 (2023) 110458. <https://doi.org/10.1016/j.asoc.2023.110458>
- [12] G.S. Ravi, K.N. Smith, P. Gokhale, F.T. Chong, Quantum computing in the cloud: analyzing job and machine characteristics, in: 2021 IEEE International Symposium on Workload Characterization (IISWC), IEEE, 2021, pp. 39–50. <https://doi.org/10.1109/IISWC53511.2021.00015>
- [13] S. Ghosh, P. Kuila, M. Bey, M. Azharuddin, Quantum-inspired gravitational search algorithm-based low-price binary task offloading for multi-users in unmanned aerial vehicle-assisted edge computing systems, Expert Syst. Appl. 263 (2025) 125762. <https://doi.org/10.1016/j.eswa.2024.125762>
- [14] E. Parker, M.J.D. Vermeer, Estimating the energy requirements to operate a cryptanalytically relevant quantum computer (2023). <https://doi.org/10.7249/WRA2427-1>
- [15] S. Ghosh, P. Kuila, Efficient offloading in disaster-affected areas using unmanned aerial vehicle-assisted mobile edge computing: a gravitational search algorithm-based approach, Int. J. Disaster Risk Reduct. 97 (2023) 104067. <https://doi.org/10.1016/j.ijdr.2023.104067>
- [16] M. Bey, P. Kuila, B.B. Naik, S. Ghosh, Quantum-inspired particle swarm optimization for efficient IoT service placement in edge computing systems, Expert Syst. Appl. 236 (2024) 121270. <https://doi.org/10.1016/j.eswa.2023.121270>
- [17] H.T. Nguyen, P. Krishnan, D. Krishnaswamy, M. Usman, R. Buyya, Quantum Cloud Computing: A Review, Open Problems, and Future Directions, arXiv preprint arXiv:2404.11420 (2024).
- [18] T. Alexander, N. Kanazawa, D.J. Egger, L. Capelluto, C.J. Wood, A. Javadi-Abhari, D.C. McKay, Qiskit pulse: programming quantum computers through the cloud with pulses, Quantum Sci. Technol. 5 (4) (2020) 044006. <https://doi.org/10.1088/2058-9565/aba404>
- [19] IBM Quantum—quantum.ibm.com, 2024, (<https://quantum.ibm.com/>). [Accessed 24-11-2024].
- [20] Quantum Inspire—quantum-inspire.com, 2024, (<https://www.quantum-inspire.com/>). [Accessed 24-11-2024].
- [21] Microsoft Quantum Overview—Quantum Machines | Microsoft Azure—azure.microsoft.com, 2024, (<https://azure.microsoft.com/en-us/solutions/quantum-computing/>). [Accessed 24-11-2024].
- [22] Hybrid Solver for Discrete Quadratic Models, 2020. <https://api.semanticscholar.org/CorpusID:229383444>
- [23] G.S. Ravi, K.N. Smith, P. Murali, F.T. Chong, Adaptive job and resource management for the growing quantum cloud, in: 2021 IEEE International Conference on Quantum Computing and Engineering (QCE), IEEE, 2021, pp. 301–312. <https://doi.org/10.1109/qce52317.2021.00047>
- [24] R. Kaewpuang, M. Xu, D. Niyato, H. Yu, Z. Xiong, J. Kang, Stochastic qubit resource allocation for quantum cloud computing, in: NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium, IEEE, 2023, pp. 1–5. <https://doi.org/10.1109/noms56928.2023.10154430>
- [25] N. Ngoenriang, M. Xu, S. Supittayapornpong, D. Niyato, H. Yu, et al., Optimal Stochastic Resource Allocation for Distributed Quantum Computing, arXiv preprint arXiv:2210.02886 (2022). <https://doi.org/10.48550/arXiv.2210.02886>
- [26] S. Ghosh, P. Kuila, T. Biswas, An energy efficient offloading technique for UAV-assisted MEC using nature inspired algorithm, in: International Conference on Frontiers of Intelligent Computing: Theory and Applications, Springer, 2022, pp. 309–318. https://doi.org/10.1007/978-981-19-7524-0_28
- [27] A. Zhu, Y. Wen, Computing offloading strategy using improved genetic algorithm in mobile edge computing system, J. Grid Comput. 19 (3) (2021) 38. <https://doi.org/10.1007/s10723-021-09578-8>
- [28] N. El Menbawy, H.A. Ali, M.S. Saraya, A.M.T. Ali-Eldin, M.M. Abdelsalam, Energy-efficient computation offloading using hybrid GA with PSO in internet of robotic things environment, J. Supercomput. 79 (17) (2023) 20076–20115. <https://doi.org/10.1007/s11227-023-05387-w>
- [29] circuit (v0.19) | IBM Quantum Documentation—docs.quantum.ibm.com, 2024, (<https://docs.quantum.ibm.com/api/qiskit/0.19/circuit>). [Accessed 27-12-2024].
- [30] A. Wack, H. Paik, A. Javadi-Abhari, P. Jurcevic, I. Faro, J.M. Gambetta, B.R. Johnson, Scale, Quality, and Speed: three key attributes to measure the performance of near-term quantum computers, arXiv preprint arXiv:2110.14108 (2021). <https://doi.org/10.48550/arXiv.2110.14108>
- [31] Updating how we measure quantum quality and speed | IBM Quantum Computing Blog—ibm.com, 2024, [Accessed 19-12-2024], <https://www.ibm.com/quantum/blog/quantum-metric-layer-fidelity>
- [32] T. Chatterjee, A. Das, S.I. Mohtashim, A. Saha, A. Chakrabarti, Qurzon: a prototype for a divide and conquer-based quantum compiler for distributed quantum systems, SN Comput. Sci. 3 (4) (2022) 323. <https://doi.org/10.1007/s42979-022-01207-9>
- [33] P. Kuila, P.K. Jana, Energy efficient clustering and routing algorithms for wireless sensor networks: particle swarm optimization approach, Eng. Appl. Artif. Intell. 33 (2014) 127–140. <https://doi.org/10.1016/j.engappai.2014.04.009>
- [34] S. Ghosh, P. Kuila, Quantum GA-driven digital twin for task urgency-aware partitioning and offloading in multi UAV-aided MEC systems, Ad Hoc Netw. (2025) 103891. <https://doi.org/10.1016/j.adhoc.2025.103891>
- [35] B. Morales-Castañeda, D. Zaldivar, E. Cuevas, F. Fausto, A. Rodríguez, A better balance in metaheuristic algorithms: does it exist?, Swarm Evol. Comput. 54 (2020) 100671. <https://doi.org/10.1016/j.swevo.2020.100671>